

Parte A - ABB

Arquivo contendo a árvore e os testes:

```
#include <iostream>
```

```
using namespace std;
```

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {  
    int info;  
    Node* dir;  
    Node* esq;  
};
```

```
class BinaryTree {  
    private:  
        Node* root;  
  
    public:  
        BinaryTree() {  
            this->root = NULL;  
        }  
  
        void inserir(int x, Node* R, Node* P);  
        bool remove(int x, Node* R, Node* P);  
        void mostra(Node* R);  
        Node* getRoot() {  
            return this->root;  
        }  
};
```

```
void BinaryTree::inserir(int x, Node* R, Node* P) {  
    if (R == NULL) {  
        R = new Node;  
        R->info = x;  
        R->dir = NULL;  
        R->esq = NULL;
```

```

    if (this->root == NULL) {
        this->root = R;
    }
    if (P != NULL) {
        if (x < P->info) {
            P->esq = R;
        } else {
            P->dir = R;
        }
    }
    return;
}

if (x == R->info) {
    return;
}

if (x < R->info) {
    return inserir(x, R->esq, R);
} else {
    return inserir(x, R->dir, R);
}
}

bool BinaryTree::remove(int x, Node* R, Node* P) {
    if (R == NULL) {
        return false;
    }

    if (R->info == x) {
        Node* aux = R;
        R = NULL;
        if (x < P->info) {
            P->esq = NULL;
        } else {
            P->dir = NULL;
        }
        delete(aux);
        return true;
    }

    if (x < R->info) {
        return remove(x, R->esq, R);
    } else {

```

```
        return remove(x, R->dir, R);
    }
}
```

```
void BinaryTree::mostra(Node* R) {
    if (R == NULL) {
        return;
    }
    mostra(R->esq);
    cout << R->info << endl;
    mostra(R->dir);
    return;
}
```

```
int main() {
```

```
    BinaryTree T = BinaryTree();
```

```
    T.inserir(50, T.getRoot(), NULL);
    T.inserir(25, T.getRoot(), NULL);
    T.inserir(10, T.getRoot(), NULL);
    T.inserir(26, T.getRoot(), NULL);
    T.mostra(T.getRoot());
    T.remove(26, T.getRoot(), NULL);
    T.mostra(T.getRoot());
```

```
}
```

Parte B - HEAP

Exercício B3 - Um heap de máximo é um heap que mantém sempre o maior valor como raiz do heap e que cada nó tem filhos menores ou iguais a ele. Ele pode ser utilizado para a implementação de filas de prioridades pois os menores valores (primeiros valores pensando em uma lista crescente) sempre estarão nos nós mais profundo à esquerda enquanto os maiores sempre estarão nos nós mais rasos tanto a direita quanto a esquerda. Assim, a eficiência de busca na lista é aumentada exponencialmente!